



IT & BUSINESS COLLEGE

Department: Computer Science

Project title: Hermes

Members:

Asylbekov Islambek – ID238715004

Anarbekov Azimbek – ID238711048

Turganaliev Diyaz – ID238715003

Bekmuhambetov Daniyar – ID238715001

Instructor:

Mirlan Nurbekov

Group:SCA-23A

Date: 06.03.2026

Contents

Abstract	2
1. Introduction	2
1.1 Background.....	2
1.2 Purpose of the Project.....	3
2. Problem Statement and Objectives	3
2.1 Problem Statement.....	3
2.2 Project Objectives.....	4
3. System Requirements and Constraints.....	4
3.1 Functional Requirements.....	4
3.2 Non-Functional Requirements	4
3.3 Hardware Requirements	5
3.4 Software Requirements	5
4. System Design.....	5
4.1 System Architecture.....	5
4.2 Database Design	6
4.3 Mobile Application Design	7
5. Implementation.....	7
5.1 Backend Implementation.....	7
5.2 Frontend Implementation	8
5.3 Database Implementation	9
6. Testing and Results.....	10
6.1 Authentication Testing.....	10
6.2 API Communication Testing	10
6.3 Booking System Testing.....	11
6.4 User Interface Testing.....	11
7. Discussion.....	11
8. Conclusion.....	13
9. References	15

Abstract

The rapid growth of mobile technologies has significantly transformed modern transportation services. In recent years, shared mobility platforms have become increasingly popular as an alternative to traditional vehicle ownership. Car-sharing systems allow users to access vehicles temporarily without the need to purchase or maintain their own cars.

This project presents the development of Hermes, a mobile car-sharing application prototype designed to simplify the vehicle rental process through a mobile platform. The application allows users to register, verify their identity, browse available cars, select pickup locations, and reserve vehicles directly from their smartphones.

The Hermes system integrates several technologies, including Flutter for mobile development, Java for backend services, and PostgreSQL for database management. Communication between the mobile application and the backend server is implemented using RESTful API endpoints.

The purpose of the Hermes prototype is to demonstrate how modern software technologies can be combined to create a functional car-sharing system. The project illustrates the full development process, including system analysis, design, implementation, testing, and documentation.

1. Introduction

1.1 Background

Urban transportation has undergone significant changes in recent years due to the rapid development of digital technologies. Many cities face challenges related to traffic congestion, high vehicle ownership costs, and limited parking space. As a result, alternative transportation solutions have become increasingly important.

One of the most promising solutions is the concept of car-sharing systems. Car-sharing platforms allow users to temporarily rent vehicles when needed instead of owning a car. This approach reduces transportation costs for users and helps decrease the number of privately owned vehicles on the road.

Modern car-sharing services rely heavily on mobile applications that allow users to quickly locate and reserve vehicles. These applications typically integrate several technological components including mobile interfaces, backend servers, databases, and location services.

1.2 Purpose of the Project

The purpose of the Hermes project is to develop a prototype of a mobile carsharing system that demonstrates the basic functionality of modern car rental platforms.

The main goals of the project include:

- Designing a user-friendly mobile application
- Implementing a secure authentication system
- Creating a vehicle inventory system
- Implementing a booking and rental process
- Managing vehicle availability
- Demonstrating communication between frontend and backend systems

The Hermes prototype provides a simplified version of a real-world carsharing service while demonstrating key software engineering concepts.

2. Problem Statement and Objectives

2.1 Problem Statement

In many urban areas, access to transportation is often inefficient due to the high costs associated with vehicle ownership. Purchasing a car requires significant financial investment, including maintenance, fuel, insurance, and parking expenses.

Traditional car rental services also present several limitations. Many rental companies require users to complete lengthy registration procedures, provide physical documents, and visit physical offices to obtain vehicles. These processes can be inconvenient for users who require quick access to transportation.

Additionally, the lack of flexible rental options can make it difficult for users to rent vehicles for short periods of time.

Car-sharing systems address these challenges by providing digital platforms that allow users to locate and rent vehicles through mobile applications. However, developing such systems requires the integration of multiple technological components including authentication systems, booking logic, and database management.

The Hermes project aims to demonstrate how such a system can be implemented using modern software development tools.

2.2 Project Objectives

The main objectives of the Hermes project are listed below.

Primary Objectives

- Develop a mobile application for vehicle rental services
- Implement secure user authentication • Design a vehicle inventory system
- Implement a vehicle booking process
- Provide location-based pickup points

Secondary Objectives

- Demonstrate integration between frontend and backend systems
- Provide a user-friendly interface
- Implement basic rental tracking functionality
- Store and manage system data in a relational database

3. System Requirements and Constraints

3.1 Functional Requirements

Functional requirements describe the actions that the system must perform.

The Hermes application must allow users to perform the following operations:

User Management

- Register a new account
- Log into the system
- Update personal profile information
- Upload driver license information

Vehicle Management

- View a list of available vehicles
- View detailed vehicle information
- Search for vehicles

Booking System

- Select a vehicle
- Choose a pickup location
- Reserve a vehicle
- Track rental duration

3.2 Non-Functional Requirements

Non-functional requirements define the quality and performance characteristics of the system.

Important non-functional requirements include:

Performance

- The application should respond quickly to user requests
- API requests should return results efficiently

Security

- User authentication must be secure
- Sensitive information must be protected

Usability

- The user interface must be simple and intuitive
- Navigation between screens should be clear and logical

Reliability

- The system should operate without frequent crashes or errors

3.3 Hardware Requirements

The Hermes application requires the following hardware components:

For development:

- Personal computer
 - Android development environment
 - Internet connection
- ### **For users:**
- Smartphone with Android operating system
 - Internet connectivity

3.4 Software Requirements

The project uses several software technologies:

Frontend

Flutter framework
Dart programming language

Backend

Java programming language
REST API architecture

Database

PostgreSQL database system

Development Tools

Android Studio
GitHub for version control
Figma for interface design

4. System Design

4.1 System Architecture

The Hermes system follows a **client-server architecture**.

The system is divided into three main components:

Mobile Application

Backend Server

Database System

The mobile application acts as the client interface used by end users. It sends requests to the backend server using HTTP requests. The backend server processes these requests and retrieves data from the PostgreSQL database.

4.2 Database Design

The database structure stores information related to users, vehicles, and bookings.

The main database entities include:

Users

- user_id
- name
- email
- password
- driver license information

Cars

- car_id
- car_name
- car_type
- price_per_hour
- availability_status

Bookings

- booking_id
- user_id
- car_id
- booking_start_time
- booking_end_time

Locations

- location_id
- location_name
- coordinates

4.3 Mobile Application Design

The mobile application includes several screens that allow users to interact with the system.

Main application screens include:

Authentication Screens

- Login page
- Registration page

Main Application Screens

- Home page
- Vehicle list page
- Vehicle details page
- Booking page

User Profile Screens

- Profile information page
- Driver license verification page

The design of the interface prioritizes usability and simplicity.

5. Implementation

The implementation stage represents the practical phase of the Hermes project where the system design is transformed into a working application. During this stage, the development team implemented both the backend infrastructure and the mobile frontend application. The goal of this stage was to create a functional prototype that demonstrates the core functionality of a car-sharing system.

The implementation process required careful coordination between frontend and backend components. The mobile application communicates with the backend server through REST API endpoints, allowing the system to exchange data related to users, vehicles, and bookings.

5.1 Backend Implementation

The backend of the Hermes system was implemented using the **Java programming language**. The backend is responsible for processing requests from the mobile application, handling authentication, managing vehicle information, and storing data in the PostgreSQL database. One of the key components of the backend implementation is the **authentication system**. Users must create an account and log into the system before accessing the main features of the application. Authentication is implemented using **JSON Web Tokens (JWT)**. After successful login, the

backend generates a token that is stored on the user's device and used to authenticate future requests.

Another important part of the backend is the **vehicle inventory management system**. The server maintains a list of vehicles that are available for rental.

Each vehicle contains information such as:

- Vehicle name
- Vehicle type
- Rental price
- Availability status
- Vehicle image

The backend also implements the **booking logic**. When a user selects a car and initiates the booking process, the backend verifies that the vehicle is available. If the car is available, the system creates a booking record in the database and updates the vehicle's status to prevent other users from renting the same vehicle simultaneously.

Additionally, the backend manages **rental duration tracking**. The system records the start time of the booking and calculates the total rental duration when the vehicle is returned.

5.2 Frontend Implementation

The frontend of the Hermes application was developed using the **Flutter framework**, which allows developers to build mobile applications for Android and iOS using a single codebase.

Flutter was selected for this project because it provides a modern UI framework, fast development cycle, and strong community support. The mobile application was designed to provide a simple and intuitive user interface.

Several important screens were implemented during the frontend development stage.

Authentication Screens

These screens allow users to create accounts and log into the application.

Main Application Screens

The home screen displays important information such as the user profile, account balance, pickup location, and a list of available vehicles.

Vehicle Selection Screen

Users can browse available cars and view detailed information about each vehicle.

Profile Screen

Users can update personal information and upload driver license data required for verification.

The frontend communicates with the backend server using HTTP requests. The mobile application sends requests to the API endpoints and processes responses received from the server.

5.3 Database Implementation

The Hermes system uses **PostgreSQL** as its database management system. PostgreSQL was selected because it is a powerful open-source relational database that provides strong data integrity and performance.

The database stores all important information related to the application. Several tables were created to organize system data.

Main database tables include:

Users Table

Stores user account information including name, email, password, and driver license details.

Cars Table

Stores vehicle information such as car model, rental price, vehicle type, and availability status.

Bookings Table

Stores information about rental transactions including which user rented which vehicle and the duration of the rental.

Locations Table

Stores predefined pickup locations that users can select when booking a vehicle.

The database interacts with the backend server through SQL queries that allow data to be inserted, retrieved, and updated as necessary.

6. Testing and Results

Testing is an essential stage of software development that ensures the reliability and functionality of the system. During the testing phase of the Hermes project, the development team evaluated different parts of the system to ensure that all components work together correctly.

The main goal of testing was to verify that the application behaves as expected and that users can perform all required actions without errors.

6.1 Authentication Testing

Authentication testing was performed to ensure that the login and registration system works correctly.

The following scenarios were tested:

- Creating a new user account
- Logging into the system using valid credentials
- Handling incorrect login attempts
- Token generation and storage
- Accessing protected API endpoints

The testing results confirmed that the authentication system functions properly and that unauthorized users cannot access restricted parts of the system.

6.2 API Communication Testing

Since the Hermes application uses a client-server architecture, proper communication between the mobile application and backend server is critical.

API testing included verifying that the mobile application can successfully send requests to the backend server and receive correct responses.

The following operations were tested:

- Retrieving the list of available vehicles
- Sending booking requests
- Retrieving user profile information
- Updating user profile data

The results showed that the REST API endpoints function correctly and return appropriate data to the mobile application.

6.3 Booking System Testing

The booking system was tested to verify that vehicle reservations work correctly.

Key test scenarios included:

- Booking an available vehicle
- Preventing double booking of the same vehicle
- Updating vehicle availability status
- Recording booking information in the database

Testing confirmed that once a vehicle is booked, its availability status changes and the system prevents other users from booking the same vehicle simultaneously.

6.4 User Interface Testing

User interface testing focused on evaluating the usability and responsiveness of the mobile application.

The following aspects were tested:

- Navigation between application screens
- Button functionality
- Form input validation
- Display of vehicle information

The results indicated that the user interface is responsive and easy to navigate.

7. Discussion

The Hermes project demonstrates how modern software technologies can be combined to create a functional prototype of a car-sharing platform. The successful integration of a mobile application, backend services, and database management allowed the development team to simulate the basic functionality of real-world carsharing systems.

One of the main strengths of the Hermes system is its **modular architecture**. The separation between the mobile frontend and the backend server provides clear structural organization and improves system maintainability. This architectural approach allows different components of the system to be developed and modified independently, which is particularly beneficial for future system expansion and scalability.

Another important aspect of the project is the use of **modern development frameworks and tools**. The Flutter framework enables efficient cross-platform mobile development, while REST API communication provides a flexible and scalable way for the mobile application to interact with the backend server. These

technologies significantly simplify the development process and allow developers to build complex mobile systems within a relatively short development period.

In addition, the project demonstrates the practical implementation of several important software engineering concepts, including authentication systems, database management, client-server communication, and modular system architecture. The development process provided valuable experience in integrating different technological components into a single functional system.

However, despite the successful development of the prototype, several **limitations of the current system** should be considered.

First, the Hermes application currently operates as a **simplified prototype**, which means that some features expected in a production-level car-sharing system are not yet implemented. For example, the current version of the system does not support real payment processing. In real car-sharing platforms, users must be able to complete secure financial transactions through payment gateways such as credit card systems or digital wallets.

Another limitation of the current system is the absence of **real-time vehicle tracking**. Many commercial car-sharing services use GPS technology to display the exact location of available vehicles on a map, allowing users to locate the nearest car quickly. In the Hermes prototype, pickup locations are predefined rather than dynamically detected.

Additionally, the current system uses a simplified vehicle inventory model. While this approach is sufficient for demonstrating the booking logic and vehicle availability management, a real-world system would require a more advanced inventory management system capable of handling a large number of vehicles and users simultaneously.

Despite these limitations, the Hermes prototype successfully demonstrates the fundamental architecture and functionality required for a car-sharing platform.

Future improvements could significantly enhance the system and bring it closer to real-world applications. Possible improvements include:

- Integration with **GPS and real-time mapping services** to display nearby vehicles
- Implementation of **secure online payment systems** for rental transactions
- Adding **vehicle tracking functionality** for better fleet management
- Implementing **push notifications** to inform users about booking updates and rental status
- Expanding the **vehicle inventory system** to support a larger fleet of vehicles

By implementing these improvements, the Hermes system could evolve from a prototype into a more advanced platform capable of supporting real-world car-sharing operations.

8. Conclusion

The Hermes project demonstrates the development of a prototype mobile carsharing application that integrates modern mobile development technologies with backend systems and database management. The primary objective of this project was to design and implement a functional prototype that simulates the basic functionality of a real-world car-sharing service.

Throughout the development process, several key components of the system were successfully implemented. The mobile application provides users with the ability to register, authenticate, browse available vehicles, select pickup locations, and initiate the booking process through an intuitive user interface. The backend system manages the core business logic of the application, including user authentication, vehicle availability management, and booking operations. The PostgreSQL database stores and organizes system data, ensuring that user information, vehicle records, and rental transactions can be retrieved and updated efficiently.

One of the main achievements of the Hermes project is the successful integration of multiple technologies into a single working system. The Flutter framework allowed the development of a responsive mobile interface, while the Java backend provided a reliable platform for processing application logic and managing data communication through REST API endpoints.

The project also demonstrates the importance of proper system architecture when developing modern software applications. By separating the frontend, backend, and database components, the system becomes more scalable and easier to maintain. This architecture allows additional features to be added in the future without requiring significant modifications to the existing system.

In addition to technical implementation, the project provided valuable experience in software development practices such as system analysis, interface design, backend development, database design, and testing. Working on the Hermes system allowed the development team to better understand the process of building a complete software system from initial concept to final prototype.

Despite the successful development of the prototype, several limitations remain. The current system does not include real payment processing, real-time GPS vehicle tracking, or advanced vehicle management systems. These features are

commonly found in commercial car-sharing platforms and would be necessary for a fully functional production system.

Future improvements to the Hermes application may include:

- integration of real-time GPS tracking for vehicles
- implementation of secure online payment systems
- expansion of the vehicle inventory system
- implementation of push notifications for booking updates
- integration with real map services for location detection

By implementing these improvements, the Hermes prototype could evolve into a more advanced system capable of supporting real-world car-sharing operations.

In conclusion, the Hermes project successfully demonstrates the development of a mobile car-sharing platform prototype. The system integrates mobile application development, backend services, and database technologies into a unified architecture that supports the core functionality of vehicle rental systems. The results of this project confirm that modern software development tools provide effective solutions for building scalable and user-friendly mobility applications.

9. References

- Turoń, K.** (2023) Car-Sharing Systems in Smart Cities: A Review of the Most Important Issues Related to the Functioning of the Systems in Light of Scientific Research. *Smart Cities*, 6(2), pp. 796–808.
- Nansubuga, B.** (2021) Carsharing: A Systematic Literature Review and Research Agenda. *Journal of Services Marketing*, 32(6), pp. 55–70.
- Tao, Z.** (2021) Research on Travel Behavior with Car Sharing under Urban Transportation Systems. *Mathematical Problems in Engineering*. Available at: [suspicious link removed] (Accessed: 13 April 2026).
- Vătămănescu, E.M.** (2024) Connecting Smart Mobility and Car Sharing: A Systematic Literature Review. *Journal of Cleaner Production*. Available at: <https://www.sciencedirect.com/journal/journal-of-cleaner-production> (Accessed: 13 April 2026).
- Flutter** (2025) *Flutter Documentation*. Available at: <https://docs.flutter.dev> (Accessed: 5 March 2026).
- Google** (2025) *Flutter SDK Overview*. Available at: <https://flutter.dev> (Accessed: 5 March 2026).
- Stošović, S., Stefanović, D., Bogdanović, M. and Vukotić, N.** (2022) The Use of the Flutter Framework in the Development Process of Hybrid Mobile Applications. *Knowledge International Journal*, 54(4), pp. 581–587.
- Kinari, S.A. and Funabiki, N.** (2025) A Guided Self-Study Platform for Flutter Cross-Platform Mobile Programming. *Computers*, 14(2). Available at: <https://www.mdpi.com/journal/computers> (Accessed: 13 April 2026).
- Flutter Team** (2025) *Flutter Architectural Overview*. Available at: <https://docs.flutter.dev/resources/architectural-overview> (Accessed: 5 March 2026).
- Wikipedia** (2026) *Flutter (software)*. Available at: [https://en.wikipedia.org/wiki/Flutter_\(software\)](https://en.wikipedia.org/wiki/Flutter_(software)) (Accessed: 5 March 2026).